



ESCOLA POLITÉCNICA · PUCRS · SISTEMAS DISTRIBUÍDOS

RabbitMQ

Mensageria com AMQP: arquitetura, garantias de entrega e ordenação

Arthur Real Sanchotene Ferreira · Caroline da Rocha de Lima · Giovani da Silva Cancherini ·
Leonardo Vargas Schilling · Osmar Sadi Nether Filho

Prof. Fernando Dotti · Porto Alegre, junho de 2026

Agenda

Sete questões guiam nossa apresentação, da motivação até a demonstração prática.

01 O que é o RabbitMQ e por que existe

02 Arquitetura e Funcionalidades

03 Tipos de Falhas Suportadas

04 Confiabilidade: modelos formais

05 Ordenação de Mensagens

06 Implementação Interna

07 Demonstração Prática: TicketLab

■ CONTEXTO

O Problema

HTTP síncrono: uma cadeia que quebra pela parte mais fraca.

Imagine o Lollapalooza abrindo vendas às 10h: 50.000 pessoas clicando ao mesmo tempo.

```
Usuário -> API -> Pagamento (aguarda 3s)
          -> Estoque (aguarda 1s)
          -> Email (aguarda 2s)
          <- responde apos 6 segundos
```

Se Pagamento demorar 30s: timeout. Usuário vê erro 500.

Se Estoque cair: toda a compra falha.

2 usuários simultâneos: overbooking possível.

O que falha

- Timeout em cascata
- Falha se qualquer serviço cair
- Overbooking
- Thread presa até o fim
- Escalar = problema

[VÍDEO OPCIONAL · SUBSTITUIR]

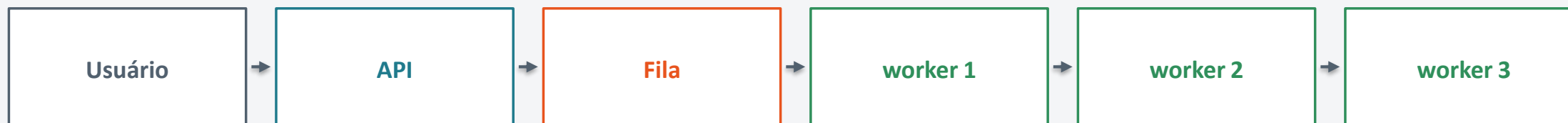
"RabbitMQ in 100 Seconds": Fireship (YouTube)

Abrir no browser antes da apresentação, reproduzir os primeiros 45s. Pausar antes do próximo slide.

■ CONTEXTO

A Solução

Desacoplar produção de consumo via broker de mensagens.



```
<- API responde imediato: { "status": "pending" }
```

■ Resposta imediata

API retorna na hora. Processamento ocorre em background.

■ Workers independentes

Cada etapa é um serviço separado. Falha num não para os outros.

■ Escalabilidade natural

Adicionar workers = aumentar throughput sem mudar a API.

O que é o RabbitMQ?

Message broker open-source: o intermediário confiável.

Message Broker

Intermediário entre quem envia e quem recebe mensagens.

Desacopla produtor de consumidor no tempo e no espaço. Não é um banco de dados: é um canal confiável.

Protocolo AMQP

Implementa AMQP 0-9-1 (Advanced Message Queuing Protocol), protocolo aberto e binário com semântica bem definida de ACK.

Também suporta STOMP, MQTT e AMQP 1.0 via plugins.

Erlang / OTP

Escrito em Erlang, linguagem criada para sistemas de telecom com 99,9999999% de disponibilidade (nine nines).

Isolamento de processos e recuperação de falhas nativos.

Onde é utilizado?

Qualquer sistema que precisa desacoplar e escalar.

E-commerce

Checkout, pagamento, estoque e notificação.
Exatamente o nosso projeto TicketLab.

Sistemas bancários

Processamento assíncrono de transações.
Auditoria e filas de conciliação.

IoT e sensores

Ingestão de eventos de milhares de dispositivos.
Fanout para múltiplos consumidores analíticos.

Microserviços

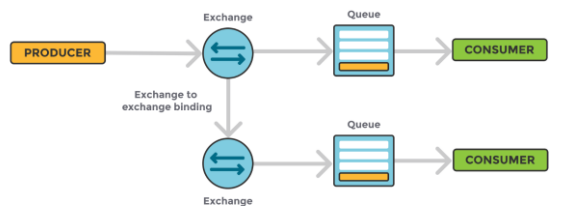
Comunicação entre bounded contexts sem acoplamento temporal.
Event-driven architecture.

Reservas / Ingressos

Controle de concorrência em estoque limitado.
Fila garante um worker por vez no estoque.

Arquitetura Interna

Producer → Exchange → Queue → Consumer.



CloudAMQP

■ Producer

Envia mensagens ao Exchange.
Não conhece as filas.

■ Exchange

Recebe e roteia via binding +
routing key. Não armazena.

■ Queue

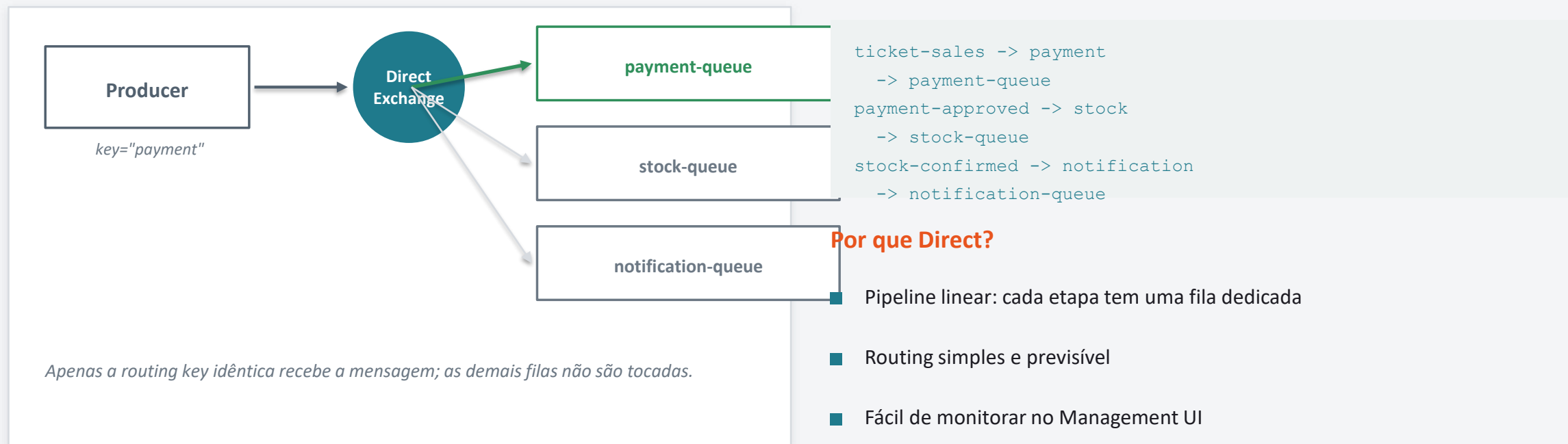
Armazena mensagens até o
consumer estar pronto.

■ Consumer

Processa e envia ACK. Só então o
broker remove da fila.

Exchange Direct

Roteamento pela chave exata: cada mensagem vai para uma única fila.

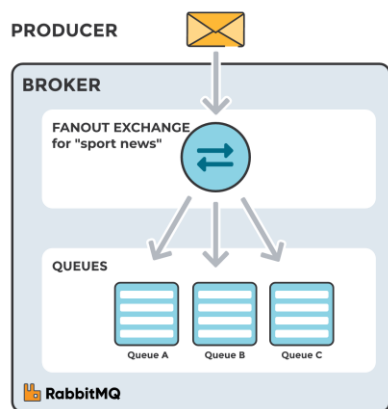


Outros Tipos de Exchange

Fanout · Topic · Headers.

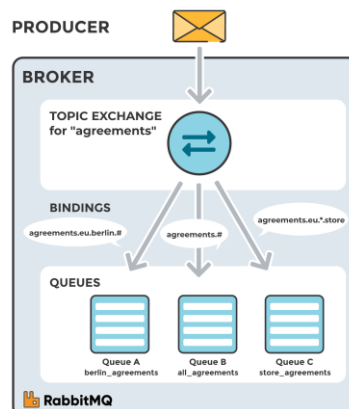
Fanout

Copia para TODAS as filas vinculadas. Ignora routing key.
Uso: broadcast de eventos (logs, auditoria, notificações globais).



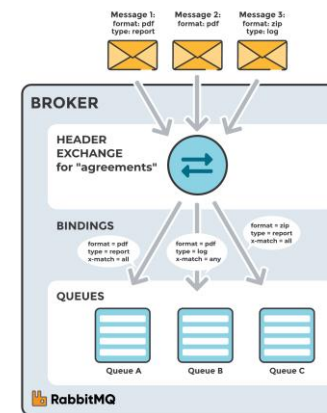
Topic

Padrão com wildcards: * (uma palavra) e # (zero ou mais).
"order.#" captura "order.created", "order.paid" ... Uso: sistemas de eventos flexíveis.



Headers

Roteia por atributos do cabeçalho AMQP.
Mais expressivo que Topic, porém mais raro na prática.
Uso: roteamento por metadados arbitrários.



Funcionalidades

O que o RabbitMQ oferece além do roteamento básico.

■ Mensagens Persistentes

`delivery_mode=PERSISTENT` grava em disco antes de confirmar. Sem isso, reinício apaga tudo.

■ Filas Durable

Metadados da fila ficam no disco. Broker reinicia: fila existe e mensagens voltam.

■ ACK Manual

Consumer confirma só após processar e salvar. Broker só remove ao receber ACK.

■ Dead Letter Exchange

NACK sem requeue envia para DLX: dead letter queue para análise e reprocessamento.

■ QoS / Prefetch

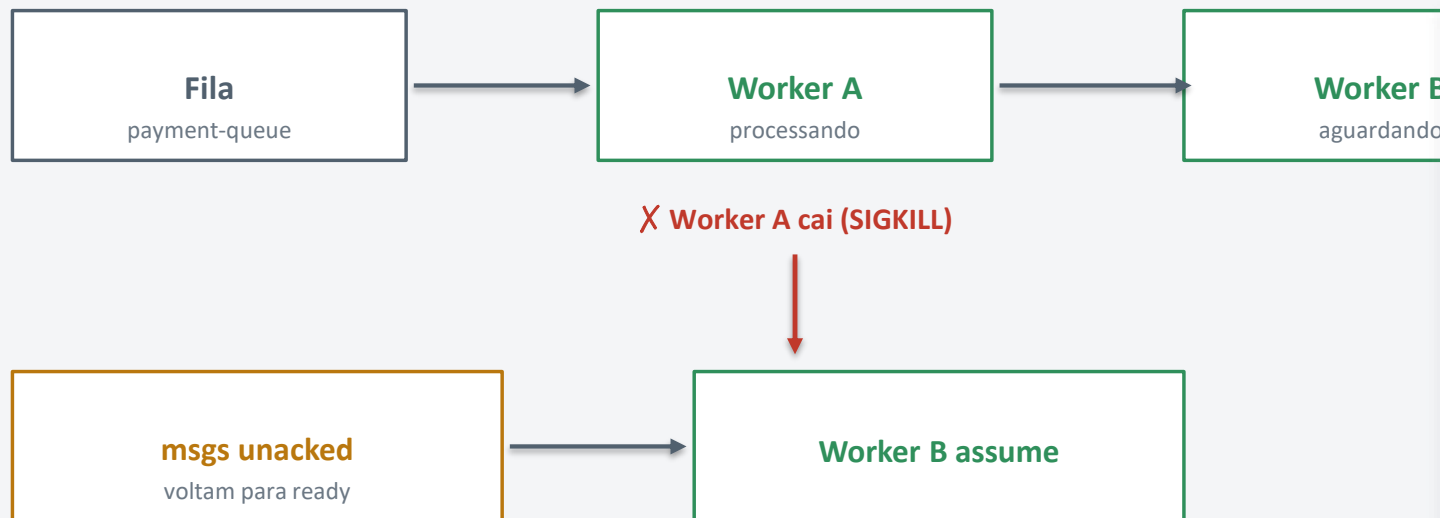
`prefetch_count` limita mensagens em voo por consumer. Backpressure automático.

■ Management UI / API

Interface web + HTTP API para monitorar filas, consumers e publicar mensagens.

Falha 1: Crash-stop do Worker

O container morre; nenhuma mensagem se perde.



Por que não perde mensagens?

- TCP disconnect detectado: RabbitMQ libera msgs unacked
- Outras msgs na fila ficam em estado ready, esperando
- Worker B (ou C) pega as mensagens liberadas
- Sem intervenção manual: é automático

```
make send-1000      # enche a fila
make kill-payment    # mata um worker abruptamente
make logs-payment    # outro worker assume as msgs
```

[GIF · SUBSTITUIR]

Terminal: kill-payment + logs-payment

Falha 2: Crash-recovery + Retry + DLQ

O broker reinicia · a lógica falha · o destino final.

Crash-recovery do Broker

- RabbitMQ reinicia (docker restart)
- Mensagens PERSISTENT + filas durable ficam no disco
- Workers reconectam automaticamente (connect_robust)
- Sistema retoma de onde parou

```
make restart-rabbit
```

[SCREENSHOT · SUBSTITUIR]

RabbitMQ UI após restart: filas com msgs intactas

Retry com Dead Letter Queue

```
msg -> worker -> falha (RuntimeError)
-> republica x-retry-count: 1
-> falha de novo: x-retry-count: 2
-> falha de novo: x-retry-count: 3
-> max_retries atingido!
-> NACK(requeue=False) -> DLX -> DLQ
```

[SCREENSHOT · SUBSTITUIR]

RabbitMQ UI: dead-letter-queue com N mensagens

■ FALHAS

Modelos de Falha: Resumo

O que suportamos vs. o que não suportamos.

✓ SUPORTADO

■ Crash-stop

Worker morre: mensagens voltam para a fila automaticamente.

■ Crash-recovery

Broker reinicia: mensagens PERSISTENT sobrevivem no disco.

■ Omissão com retry

Falhas de lógica geram DLQ após N tentativas.

■ Partição de rede

connect_robust reconecta com backoff exponencial.

✗ NÃO SUPORTADO

■ Falhas Bizantinas

Nó malicioso enviando mensagens forjadas ou corrompidas.

■ Exatamente-uma-vez nativo

At-least-once: duplicatas possíveis (tratamos com idempotência).

Confiabilidade: Fair-Loss vs Perfect Link

O que acontece com as mensagens quando o broker cai?

Fair-Loss Link

Mensagens podem se perder, mas não são inventadas.

```
exchange -> non-durable
fila -> non-durable
delivery -> NON_PERSISTENT (RAM)
```

Resultado:

- Broker reinicia: mensagens na RAM somem
- Nenhum aviso ao producer
- Garantia? Nenhuma.

Perfect Link (at-least-once)

Toda mensagem enviada é eventualmente entregue.

```
exchange -> durable
fila -> durable
delivery -> PERSISTENT (disco)
ACK -> manual
conexao -> connect_robust
```

Ressalva: at-least-once

Worker processa e falha antes do ACK: recebe a mensagem de novo.
Idempotência (próximos slides) trata isso.

BEB vs Reliable Broadcast

O producer falha no meio: o que acontece?

Best Effort Broadcast

O emissor dispara e não garante se ele mesmo falhar.

```
# publisher.py - sem publisher confirms
await exchange.publish(msg, key='payment')
# broker cai aqui -> msg perdida
```

- Enviamos 1000 mensagens
- Broker cai na mensagem 500
- Mensagens 501 a 1000 perdidas
- Producer não tem como saber

Reliable Broadcast (aprox.)

Se um processo correto entrega m, todos entregam m.

- Mensagem com PERSISTENT nunca some
- ACK manual: broker guarda até confirmar
- Worker ativo processa eventualmente

O que ainda falta para RB formal:

- Publisher confirms (sem eles = BEB)
- Se producer cai antes de publicar, não há recuperação

Uniform Reliable Broadcast

O nível mais alto: como chegamos perto.

Definição de URB: se qualquer processo (correto ou com falha) entrega m, TODOS entregam m. Requer consenso.

✗ RabbitMQ não garante URB nativo

Worker A: recebe a mensagem, processa parcialmente, salva metade no banco e falha antes do ACK.

Worker B: recebe a mesma mensagem de novo. O estado parcial do Worker A já existe: sem consenso sobre quem entregou.

✓ Nossa solução: idempotência

```
# notification-worker/worker.py
if order.status == 'confirmed':
    return # ja processado, ignora
```

Resultado: exactly-once semântico para o usuário, mesmo com at-least-once na fila.

Quorum Queues (RabbitMQ 3.8+ com Raft) aproximam o URB nativamente: próximo passo real.

Comparativo: os 5 Modelos

De Fair-Loss até URB: onde o RabbitMQ se encaixa.

Modelo	Perde msg?	Duplicatas?	Crash producer	No projeto
Fair-Loss Link	Sim	Não	n/a	Sem persistência
Perfect Link	Não	Possível	Perde msgs	PERSISTENT + ACK + durable
BEB	Possível	Não	Perde msgs	POST /batch (sem confirms)
Reliable Broadcast	Não	Possível	Perde msgs	Aprox. com persistência
Uniform RB	Não	Não	Nenhuma	Não nativo (idempotência)

Linha destacada = nossa implementação principal.

■ ORDENAÇÃO

Ordenação: FIFO com 1 Consumer

A fila garante a ordem. Um consumidor a respeita.



ACK por mensagem, sempre na mesma ordem: msg1 -> ack -> msg2 -> ack -> msg3 -> ack -> msg4 -> ack -> msg5 -> ack

Propriedade FIFO garantida pelo RabbitMQ

A especificação AMQP 0-9-1 define que mensagens publicadas com a mesma routing key e priority chegam à fila na ordem em que foram publicadas, e são entregues nessa ordem a um único consumer.

- **Mas:** com N consumers em paralelo, a ordem global de processamento NÃO é garantida (próximo slide).

Ordenação: N Consumers

Paralelo quebra a ordem global. Como resolver.



```
make scale-workers N=5 && make send-100
make db-query
# processed_at NÃO respeita created_at
```

Como garantir ordem total, se necessário

Particionamento

1 worker por event_id, como Kafka faz com partition keys. Simples e eficaz.

Sequencer

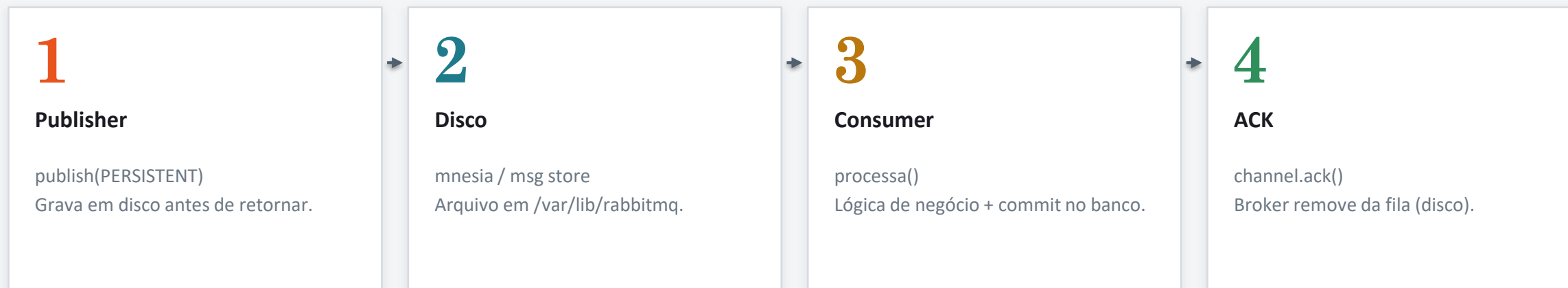
Nó central atribui número de sequência antes de publicar. Gargalo, mas garante ordem.

Kafka

Nativo: cada partição é uma fila FIFO. Workers de uma partição respeitam a ordem.

Implementação: Persistência + ACK

Como o RabbitMQ garante que mensagens não somem.



⚠ Sem ACK

Crash antes do ACK: a mensagem volta para o estado ready e é entregue de novo. Isso é o preço do at-least-once: necessário idempotência no processamento.

Implementação: Anti-Overbooking

SELECT FOR UPDATE evita a venda dupla do mesmo ingresso.

Sem controle: o que pode acontecer

- Worker A lê available_tickets = 1, reserva
- Worker B lê available_tickets = 1 ao mesmo tempo, reserva também
- **Resultado: 2 vendas, 0 ingressos (overbooking)**

Com SELECT FOR UPDATE: nossa solução

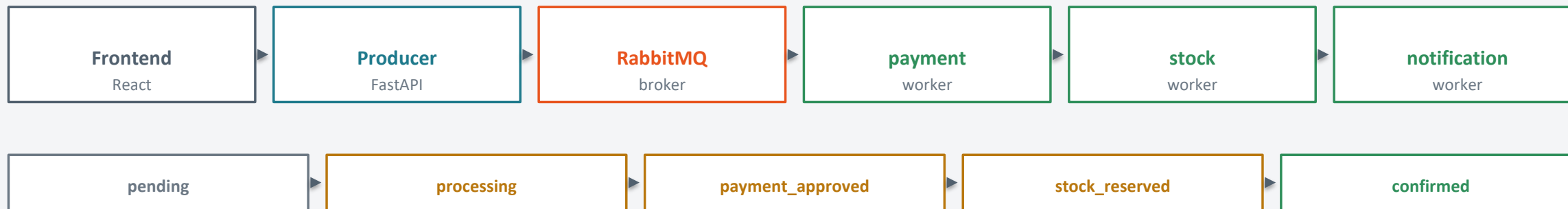
```
-- stock-worker/src/database.py
BEGIN;
SELECT * FROM events
  WHERE id = $1
  FOR UPDATE;    -- bloqueia outros workers
UPDATE events SET available_tickets -= $qty;
UPDATE orders SET status = 'stock_reserved';
COMMIT;          -- libera o lock
```

- Worker B tenta e bloqueia
- Aguarda Worker A commitar
- Se estoque zerou: recebe out_of_stock
- Sem overbooking possível

■ DEMONSTRAÇÃO

TicketLab: Sistema de Venda de Ingressos

Materializa todos os conceitos vistos: o RabbitMQ é o protagonista, não infraestrutura oculta.



Conceitos demonstrados

- Perfect Links: filas durable + PERSISTENT + ACK manual
- Crash-stop: make kill-payment, sem perda de mensagens
- Race condition: SELECT FOR UPDATE, zero overbooking
- DLQ: make send-failures (30% falha) -> make inspect-dlq
- Escalabilidade: make scale-workers N=5

[SCREENSHOT · SUBSTITUIR]

Frontend TicketLab: tela principal com eventos

localhost:3000: capturar a Home com os cards de eventos (Coldplay, Lollapalooza etc.)

Demo: Saga Coreografada

Um pedido passa por 4 etapas; cada uma é um worker independente.

1

Compra

Formulário preenchido -> POST /orders.
API retorna status: pending de imediato.

[SCREENSHOT · SUBSTITUIR]

localhost:3000/buy: formulário preenchido antes de clicar

2

Pagamento

payment-worker consome payment-queue.
Valida -> publica em payment-approved.

[SCREENSHOT · SUBSTITUIR]

localhost:3000/orders: status processing

3

Estoque

stock-worker consome stock-queue.
SELECT FOR UPDATE reserva o ingresso.
Status: stock_reserved.

[SCREENSHOT · SUBSTITUIR]

Mesma sequência, capturar em seguida

4

Confirmação

notification-worker gera ticket_code (UUID).
Verifica idempotência.
Status: confirmed. Ingresso na mão do

[SCREENSHOT · SUBSTITUIR]

localhost:3000/orders: detalhe com status confirmed

Demo: Falhas na Prática

Três cenários: o sistema é resiliente.

Dead Letter Queue

```
make send-failures  
make inspect-dlq
```

30% dos pedidos com `simulate_failure`. 3 retries, depois NACK -> DLQ.

Crash-stop Worker

```
make send-1000  
make kill-payment  
make logs-payment
```

Outro worker assume as mensagens unacked automaticamente.

Crash-recovery Broker

```
make restart-rabbit
```

Mensagens PERSISTENT no disco. Workers reconectam com `connect_robust`.

[SCREENSHOT · SUBSTITUIR]

RabbitMQ UI: dead-letter-queue com N mensagens (localhost:15672)

[SCREENSHOT · SUBSTITUIR]

Grafana: consumer count caindo de 2 para 1 (localhost:3001)

[SCREENSHOT · SUBSTITUIR]

RabbitMQ UI após restart: filas intactas (localhost:15672)

■ CONCLUSÃO

Conclusão

O que aprendemos sobre o RabbitMQ em Sistemas Distribuídos

QUESTÃO DO PROFESSOR	RESPOSTA
Onde é usado?	E-commerce, IoT, bancos, microserviços: desacoplamento temporal.
Funcionalidades?	Exchanges, durable queues, PERSISTENT, ACK, DLX, QoS, plugins.
Falhas suportadas?	Crash-stop, crash-recovery, retry+DLQ. Não suporta Byzantine.
Confiabilidade?	Perfect Links (at-least-once) + aproximação de Reliable Broadcast.
Ordenação?	FIFO com 1 consumer. N consumers requer particionamento.
Implementação?	AMQP 0-9-1, Erlang/Mnesia, protocolo ACK, SELECT FOR UPDATE.

Próximos passos para produção

- Publisher confirms: Reliable Broadcast formal (saindo do BEB)
- Outbox Pattern: zero perda durante partição producer-broker
- Quorum Queues (Raft): aproximação de URB nativa no RabbitMQ 3.8+
- Kafka + partition keys: ordem total por evento

Obrigado. Ficamos à disposição para perguntas.